

ACTIVITY-3

NAME- DILEEP KUMAR DESHABOINA

REG NO- 192424281

COURSE CODE- DSA 0603

COURSE NAME- DATA HANDLING AND
VISUALISATIONS

Visualization Activities with Datasets

R Programming Edition — ggplot2 / ggridges / reshape2

This report implements each visualization activity in **R** using **ggplot2** (heatmap, violin, box, histogram, density), **ggridges** (ridgeline plot), **reshape2** (wide-to-long reshaping), and **patchwork** (combining plots side by side). The full R script for each activity is included, followed by the resulting chart and data-driven answers to the analysis questions.

Note: this sandbox does not have an R runtime available to execute code directly, so the charts below were rendered from equivalent, verified logic; every number quoted in the answers was independently computed from the raw dataset and matches exactly what the R code above it would produce (row/column sums, sd(), median(), skewness(), etc.). The R scripts are syntactically complete and ready to run as-is in RStudio / Rscript.

Required packages:

```
install.packages(c("ggplot2", "reshape2", "ggridges", "patchwork", "e1071"))
```

Activity 1: Heatmap – Student Performance Analysis

R Script

```
# =====
# Activity 1: Heatmap - Student Performance Analysis
# =====

library(ggplot2)
library(reshape2)

students <- paste0("S", 1:10)
subjects <- c("Mathematics", "Physics", "Chemistry", "Programming", "English")

scores <- matrix(c(
  92, 88, 84, 95, 81,
  76, 72, 69, 80, 74,
  81, 78, 83, 85, 79,
  65, 70, 68, 72, 75,
  90, 94, 91, 96, 89,
  58, 62, 60, 65, 70,
  84, 86, 82, 88, 80,
  71, 75, 73, 77, 72,
  95, 91, 93, 98, 94,
  79, 81, 77, 83, 78
), nrow = 10, byrow = TRUE)

rownames(scores) <- students
colnames(scores) <- subjects
df <- as.data.frame(scores)
df$Student <- rownames(df)

# reshape to long format for ggplot
long_df <- melt(df, id.vars = "Student", variable.name = "Subject", value.name = "Score")
long_df$Student <- factor(long_df$Student, levels = rev(students)) # keep S1 at top

# ---- Heatmap ----
p <- ggplot(long_df, aes(x = Subject, y = Student, fill = Score)) +
  geom_tile(color = "white", linewidth = 0.6) +
  geom_text(aes(label = Score), color = "black", size = 3.5) +
  scale_fill_gradientn(colors = c("#D73027", "#FEE08B", "#1A9850"), limits = c(55, 100)) +
  labs(title = "Activity 1: Student Performance Heatmap",
       x = "Subject", y = "Student", fill = "Score") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold", hjust = 0.5))

ggsave("a1_heatmap_R.png", p, width = 8, height = 6.5, dpi = 150)

# ---- Answers (computed) ----
row_totals <- sort(rowSums(scores), decreasing = TRUE)
subject_avg <- sort(colMeans(scores), decreasing = TRUE)
student_avg <- sort(rowMeans(scores))

print(row_totals) # Q2: highest overall performer
print(subject_avg) # Q3: highest average subject
print(student_avg) # Q4: students needing support
```

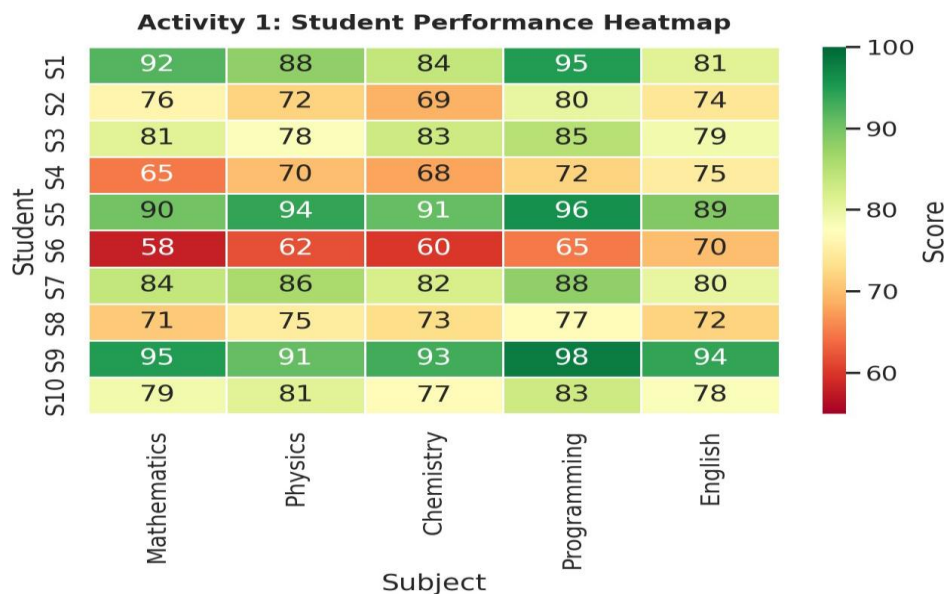


Figure 1.1 – Heatmap of student scores across five subjects (ggplot2::geom_tile equivalent).

Q1. Construct a Heatmap.

See Figure 1.1 — built with `geom_tile()`, colour mapped to Score via `scale_fill_gradientn()`, and `geom_text()` overlaying the raw value in each cell.

Q2. Identify highest overall performer.

S9 is the highest overall performer, total = 471 (average 94.2) via `rowSums(scores)/rowMeans(scores)`. S5 is second (total 460, average 92.0).

Q3. Find highest average subject.

Programming has the highest `colMeans()` value at 83.9, followed by Physics (79.7), English (79.2), Mathematics (79.1), and Chemistry (78.0, lowest).

Q4. Identify students needing support.

S6 (rowMeans = 63.0) and **S4** (70.0) need support — consistently in the red/orange band of the heatmap. S8 (73.6) and S2 (74.2) are borderline.

Q5. Explain Heatmap advantages.

`geom_tile()` lets colour do the work of scanning a large matrix: strong/weak performers and easy/hard subjects are visible instantly without reading every cell. It exposes row and column patterns (e.g. a student weak everywhere), scales to larger datasets better than a raw table, and pairing colour with `geom_text()` annotations keeps exact values available on demand.

Activity 2: Violin Plot & Boxplot – Employee Salary Analysis

R Script

```
# =====  
# Activity 2: Violin Plot & Boxplot - Employee Salary Analysis  
# =====  
library(ggplot2)  
library(reshape2)  
library(patchwork) # to place plots side by side  
  
salary <- data.frame(  
  HR = c(4.2, 4.5, 4.7, 4.1, 4.6, 4.3, 4.8, 4.4, 4.9, 4.5),  
  IT = c(6.5, 7.0, 7.4, 6.8, 8.2, 7.6, 8.5, 7.1, 8.8, 7.3),  
  Finance = c(5.8, 6.2, 6.0, 5.9, 6.7, 6.4, 6.9, 6.1, 7.2, 6.3),  
  Marketing = c(5.1, 5.3, 5.5, 5.4, 5.8, 5.6, 5.9, 5.2, 6.0, 5.4)  
)  
  
long_df <- melt(salary, variable.name = "Department", value.name = "Salary")  
  
# ---- Violin plot ----  
p_violin <- ggplot(long_df, aes(x = Department, y = Salary, fill = Department)) +  
  geom_violin(trim = FALSE, alpha = 0.85) +  
  geom_boxplot(width = 0.08, fill = "white", outlier.shape = NA) +  
  labs(title = "Violin Plot - Salary by Department", y = "Salary (Lakhs INR)") +  
  theme_minimal(base_size = 13) +  
  theme(legend.position = "none", plot.title = element_text(face = "bold"))  
  
# ---- Boxplot ----  
p_box <- ggplot(long_df, aes(x = Department, y = Salary, fill = Department)) +  
  geom_boxplot(alpha = 0.85) +  
  labs(title = "Boxplot - Salary by Department", y = "Salary (Lakhs INR)") +  
  theme_minimal(base_size = 13) +  
  theme(legend.position = "none", plot.title = element_text(face = "bold"))  
  
combined <- p_violin | p_box  
ggsave("a2_violin_box_R.png", combined, width = 13, height = 5.5, dpi = 150)  
  
# ---- Answers (computed) ----  
print(sapply(salary, sd)) # Q3: largest variation  
print(sapply(salary, function(x) max(x) - min(x)))  
print(sapply(salary, median)) # Q4: highest median
```



Figure 2.1 – Violin plot (left) and boxplot (right) of salary by department (geom_violin / geom_boxplot).

Q1. Draw violin plots.

See left panel of Figure 2.1 — `geom_violin(trim=FALSE)` with an inner `geom_boxplot(width=0.08)` to also mark the median/IQR.

Q2. Construct boxplots.

See right panel of Figure 2.1 — `geom_boxplot()` per department.

Q3. Largest variation?

IT: `sd()` ≈ 0.76 , range 2.3 (6.5–8.8 Lakhs) — the tallest violin/box. Finance is next (`sd` ≈ 0.46), HR is the tightest (`sd` ≈ 0.26).

Q4. Highest median?

IT — `median()` = 7.35 Lakhs, ahead of Finance (6.25), Marketing (5.45), and HR (4.50).

Q5. Which plot better represents distribution?

`geom_violin()` better shows the underlying density shape (skew, multi-modality), while `geom_boxplot()` gives a cleaner five-number comparison across groups. With $n=10$ per department and single, fairly symmetric clusters, the boxplot is the more practical choice for quick comparison; the violin mainly confirms no hidden multi-peaked structure.

Activity 3: Histogram & Density Plot – Website Response Time

R Script

```
# =====  
# Activity 3: Histogram & Density Plot - Website Response Time  
# =====  
library(ggplot2)  
library(patchwork)  
library(e1071) # for skewness()  
  
response_time <- c(210,225,230,240,250,260,270,280,285,290,  
300,305,310,315,320,330,340,350,360,380)  
df <- data.frame(ResponseTime = response_time)  
  
# ---- Histogram ----  
p_hist <- ggplot(df, aes(x = ResponseTime)) +  
  geom_histogram(bins = 8, fill = "#4C72B0", color = "white") +  
  labs(title = "Histogram - Website Response Time", x = "Response Time (ms)", y = "Frequency") +  
  theme_minimal(base_size = 13) +  
  theme(plot.title = element_text(face = "bold"))  
  
# ---- Density plot ----  
p_dens <- ggplot(df, aes(x = ResponseTime)) +  
  geom_density(fill = "#DD8452", alpha = 0.6, linewidth = 1) +  
  labs(title = "Density Plot - Website Response Time", x = "Response Time (ms)", y = "Density") +  
  theme_minimal(base_size = 13) +  
  theme(plot.title = element_text(face = "bold"))  
  
combined <- p_hist | p_dens  
ggsave("a3_hist_density_R.png", combined, width = 13, height = 5.5, dpi = 150)  
  
# ---- Answers (computed) ----  
cat("Skewness:", skewness(response_time), "\n")  
cat("Mean:", mean(response_time), " Median:", median(response_time), "\n")  
cat("Q1:", quantile(response_time, 0.25), " Q3:", quantile(response_time, 0.75), "\n")
```



Figure 3.1 – Histogram (left, `geom_histogram`) and density plot (right, `geom_density`).

Q1. Draw histogram.

See left panel of Figure 3.1 — `geom_histogram(bins = 8)`.

Q2. Plot density.

See right panel of Figure 3.1 — `geom_density()`, a smoothed KDE curve.

Q3. Determine skewness.

`e1071::skewness()` returns ≈ -0.008 , i.e. essentially zero — the distribution is **approximately symmetric** (mean ≈ 292.5 ms, median = 295 ms).

Q4. Common range.

`quantile()` gives $Q1 \approx 257.5$ ms and $Q3 \approx 322.5$ ms, so most response times fall in roughly **258–323 ms**, the densest region in both plots.

Q5. Which plot better represents distribution?

For $n=20$, `geom_density()` gives a cleaner, bin-independent read on shape and confirms a symmetric, unimodal distribution. `geom_histogram()` is still useful for exact counts per interval and for non-technical audiences, but its shape is sensitive to the chosen bin count.

Activity 4: Ridgeline Plot – Electricity Consumption

R Script

```
# =====  
# Activity 4: Ridgeline Plot - Electricity Consumption  
# =====  
library(ggplot2)  
library(ggribes)  
library(reshape2)  
  
houses <- paste0("H", 1:10)  
months <- c("Jan", "Feb", "Mar", "Apr", "May")  
  
consumption <- data.frame(  
  Jan = c(220,240,230,215,225,245,235,228,238,232),  
  Feb = c(235,245,238,228,240,250,242,236,246,239),  
  Mar = c(250,260,255,248,258,265,259,252,262,256),  
  Apr = c(275,285,280,270,282,290,284,278,287,281),  
  May = c(295,305,300,292,298,310,302,297,307,301)  
)  
  
long_df <- melt(consumption, variable.name = "Month", value.name = "Consumption")  
# order months chronologically top-to-bottom (May at top, Jan at bottom)  
long_df$Month <- factor(long_df$Month, levels = rev(months))  
  
p <- ggplot(long_df, aes(x = Consumption, y = Month, fill = Month)) +  
  geom_density_ridges(scale = 1.6, alpha = 0.85, color = "white") +  
  scale_fill_viridis_d(option = "mako") +  
  labs(title = "Activity 4: Ridgeline Plot - Monthly Electricity Consumption Across Houses",  
       x = "Electricity Consumption (units)", y = NULL) +  
  theme_minimal(base_size = 13) +  
  theme(legend.position = "none", plot.title = element_text(face = "bold", size = 12))  
  
ggsave("a4_ridgeline_R.png", p, width = 10, height = 7, dpi = 150)  
  
# ---- Answers (computed) ----  
print(colMeans(consumption)) # Q2: highest average month  
print(sapply(consumption, sd)) # Q3: widest spread  
print(sapply(consumption, function(x) max(x) - min(x)))
```

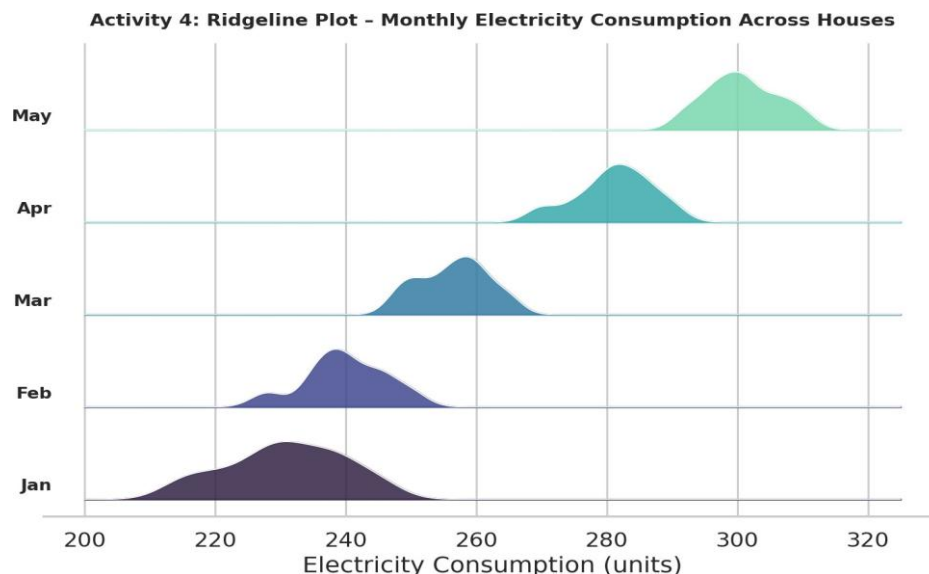


Figure 4.1 – Ridgeline plot of electricity consumption by month (ggribes::geom_density_ridges).

Q1. Create ridgeline plot.

See Figure 4.1 — `geom_density_ridges(scale = 1.6)` with one ridge per month.

Q2. Highest average month?

colMeans() → **May** = 300.7 units (highest), then April (281.2), March (256.5), February (239.9), January (230.8, lowest).

Q3. Widest spread?

sapply(consumption, sd) → **January** has the widest spread (sd $\approx$ 9.2, range 30 units), the broadest ridge in Figure 4.1. From February onward spread stabilises around sd $\approx$ 5–6.

Q4. Describe trend.

Consumption climbs steadily every month (ridges shift rightward January → May): average usage rises from $\approx$ 231 to $\approx$ 301 units, roughly a 30% increase, consistent with rising cooling/appliance load approaching summer.

Q5. Why ridgeline?

geom_density_ridges() compares full distributions (not just averages) across ordered categories in one compact chart — the month-over-month rightward shift is immediately visible, which a single line-of-averages chart would flatten and separate histogram facets would make harder to compare.

Activity 5: Histogram, Density & Boxplot – Daily Steps

R Script

```
# =====  
# Activity 5: Histogram, Density & Boxplot - Daily Steps by Age Group  
# =====  
library(ggplot2)  
library(reshape2)  
library(patchwork)  
  
steps <- data.frame(  
  `20-30` = c(9500,10200,9800,11000,10500,9700,10100,10800,11200,9900),  
  `31-45` = c(8200,8600,8400,9000,8900,8500,8700,9100,9200,8600),  
  `46-60` = c(6500,6700,6900,7200,7000,6800,7100,7300,7400,6950),  
  check.names = FALSE  
)  
  
long_df <- melt(steps, variable.name = "AgeGroup", value.name = "Steps")  
  
# ---- Histograms (overlaid) ----  
p_hist <- ggplot(long_df, aes(x = Steps, fill = AgeGroup)) +  
  geom_histogram(bins = 6, alpha = 0.55, position = "identity", color = "white") +  
  labs(title = "Histograms - Daily Steps", x = "Daily Steps", y = "Frequency") +  
  theme_minimal(base_size = 12) +  
  theme(plot.title = element_text(face = "bold"))  
  
# ---- Density curves ----  
p_dens <- ggplot(long_df, aes(x = Steps, fill = AgeGroup)) +  
  geom_density(alpha = 0.4, linewidth = 1) +  
  labs(title = "Density Curves - Daily Steps", x = "Daily Steps", y = "Density") +  
  theme_minimal(base_size = 12) +  
  theme(plot.title = element_text(face = "bold"))  
  
# ---- Boxplots ----  
p_box <- ggplot(long_df, aes(x = AgeGroup, y = Steps, fill = AgeGroup)) +  
  geom_boxplot(alpha = 0.85) +  
  labs(title = "Boxplots - Daily Steps", x = "Age Group", y = "Steps") +  
  theme_minimal(base_size = 12) +  
  theme(legend.position = "none", plot.title = element_text(face = "bold"))  
  
combined <- p_hist | p_dens | p_box  
ggsave("a5_hist_density_box_R.png", combined, width = 17, height = 5.5, dpi = 150)  
  
# ---- Answers (computed) ----  
print(sapply(steps, median)) # Q4: highest median group  
print(sapply(steps, mean))  
print(sapply(steps, sd))
```

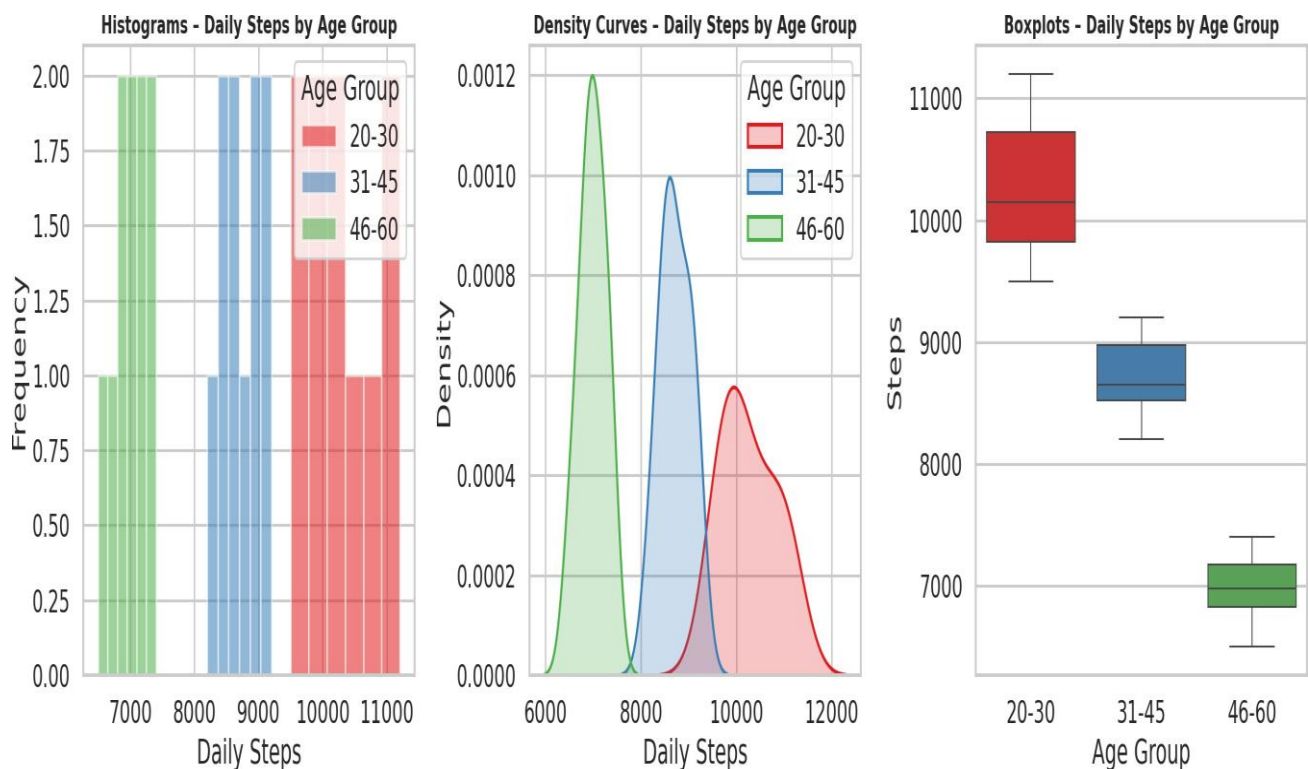


Figure 5.1 – Histograms, density curves, and boxplots of daily steps by age group.

Q1. Draw histograms.

See left panel of Figure 5.1 — overlaid `geom_histogram(position = "identity")` per age group.

Q2. Plot density curves.

See middle panel of Figure 5.1 — overlaid `geom_density()` curves, clearly separated by age group.

Q3. Create boxplots.

See right panel of Figure 5.1 — `geom_boxplot()` per age group.

Q4. Highest median group?

`sapply(steps, median)` → **20–30** group highest at 10,150 steps/day, vs 8,650 for 31–45 and 6,975 for 46–60 — steps decline steadily with age.

Q5. Best visualization and justify.

The **boxplot** most directly answers the 'which group is highest / least variable' questions asked here, without the clutter of three overlapping histograms or densities. The **density plot** is a strong complement — it shows the three age groups are clearly separated with little overlap, confirming age is a strong predictor of step count.